AI Code Review Agent

Step 2 – Implementation Progress

1. Updated System Description

The AI Code Review Agent has been partially implemented in Python. The current version of the project already includes the main project structure, dependency configuration, documentation files, and the core implementation of the intelligent review workflow.

The system currently operates through a command-line interface where the user provides Python source code for analysis. After receiving the code, the intelligent agent coordinates several analysis modules and generates structured feedback related to syntax correctness and software complexity.

At the current implementation stage, the project already contains:

- a modular project structure,
- external tool integrations,
- syntax analysis functionality,
- complexity analysis functionality,
- intelligent agent workflow coordination,
- dependency management configuration,
- initial application execution workflow.

The implementation focuses on modularity, maintainability, readability, and independent testing of system components.

2. Programming Concepts Actually Used

Several important software engineering and programming concepts are currently used in the implementation of the project.

Object-Oriented Programming

The project uses object-oriented programming principles to separate the responsibilities of different components. Independent classes are implemented for:

- syntax analysis,
- complexity analysis,
- intelligent workflow coordination.

This improves maintainability and code organization.

Modular Software Architecture

The project is divided into multiple independent modules and directories such as:

- `src/tools`,
- `src/agent`,
- `tests`,
- `docs`.

This structure improves readability, debugging, testing, and future expansion of the system.

External Library Integration

The system integrates external analysis tools and libraries including:

- Python built-in `ast` module,
- Pylint,
- Radon.

These tools are integrated through dedicated Python modules inside the `tools` directory.

File and Data Handling

The system processes Python source code as text input and transfers the data between modules during execution. Structured dictionary-based outputs are used to preserve consistency between different system components.

Exception Handling

Exception handling is implemented using try-except blocks to safely manage syntax errors and invalid source code during analysis.

This improves system reliability and prevents unexpected application crashes.

Version Control

Git and GitHub are actively used during development. The repository currently contains:

- multiple implementation commits,
- modular project organization,
- dependency configuration,
- documentation updates,
- implementation history.

This demonstrates visible project evolution during development.

3. How These Concepts Are Applied

Object-oriented programming is applied by implementing separate classes for each major responsibility in the system. For example, the syntax analysis logic is separated from the complexity analysis logic and from the intelligent review workflow.

The modular architecture separates the project into independent components that can be tested and extended separately. This allows easier maintenance and future integration of additional analysis tools.

External library integration is implemented through wrapper modules that standardize tool outputs before they are processed by the intelligent agent. This improves compatibility between system components.

Structured dictionary outputs are used to normalize returned data from different tools. This simplifies result processing and report generation.

Exception handling mechanisms are applied during syntax analysis to safely capture and report invalid Python code without interrupting program execution.

Git and GitHub are used to maintain version history and demonstrate incremental implementation progress through meaningful commits.

4. Tool Integration Description

The current implementation integrates several analysis tools into the intelligent agent workflow.

The syntax analysis module uses Python's built-in `ast` library to validate source code structure and detect syntax errors. The tool processes source code input and returns structured validation results.

The style analysis module integrates Pylint. This tool is designed to analyse formatting issues, variable naming conventions, and maintainability-related problems in Python code.

The complexity analysis module integrates Radon to measure cyclomatic complexity and identify functions that may be difficult to maintain.

The intelligent review agent coordinates communication between all analysis modules. The agent collects outputs from different tools and combines them into a unified structured result.

The current implementation also includes a main execution workflow where the agent processes sample Python code and returns analysis results during runtime.